HIRAL SARKAR · PGA 45 CAPSTONE

Resolvia, from break to verified resolution

I built Resolvia to take a post-trade break from detection to a verified fix. This guide walks through how I built the data, the models and the agents, and what the results show. Every idea has a picture.

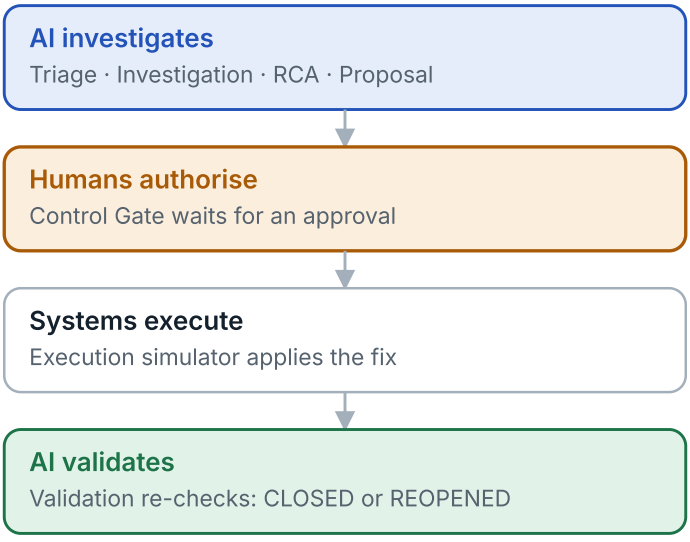
AI investigates · Humans authorise · Systems execute · AI validates

Idea · Architecture · Data · Modeling · Agents

01 · THE IDEA

Matching finds the break. Resolvia resolves it.

A reconciliation engine asks “does A equal B?” and raises a break when it doesn't. That part is solved: DTCC reports 98.4% same-day equity matching. The hard, manual part comes next: finding evidence, working out why, deciding the fix, getting sign-off and proving it worked. Resolvia automates that chain for cash-equity breaks, and keeps a human in charge of anything that moves money.



The one line to remember. Blue is AI, amber is human control, green is verification.

Scope. Indian cash equities (T+1, INR) · 4 break families · 20 root causes · a corporate-action specialist case. Out of scope: derivatives, FX, lending, real payment systems.



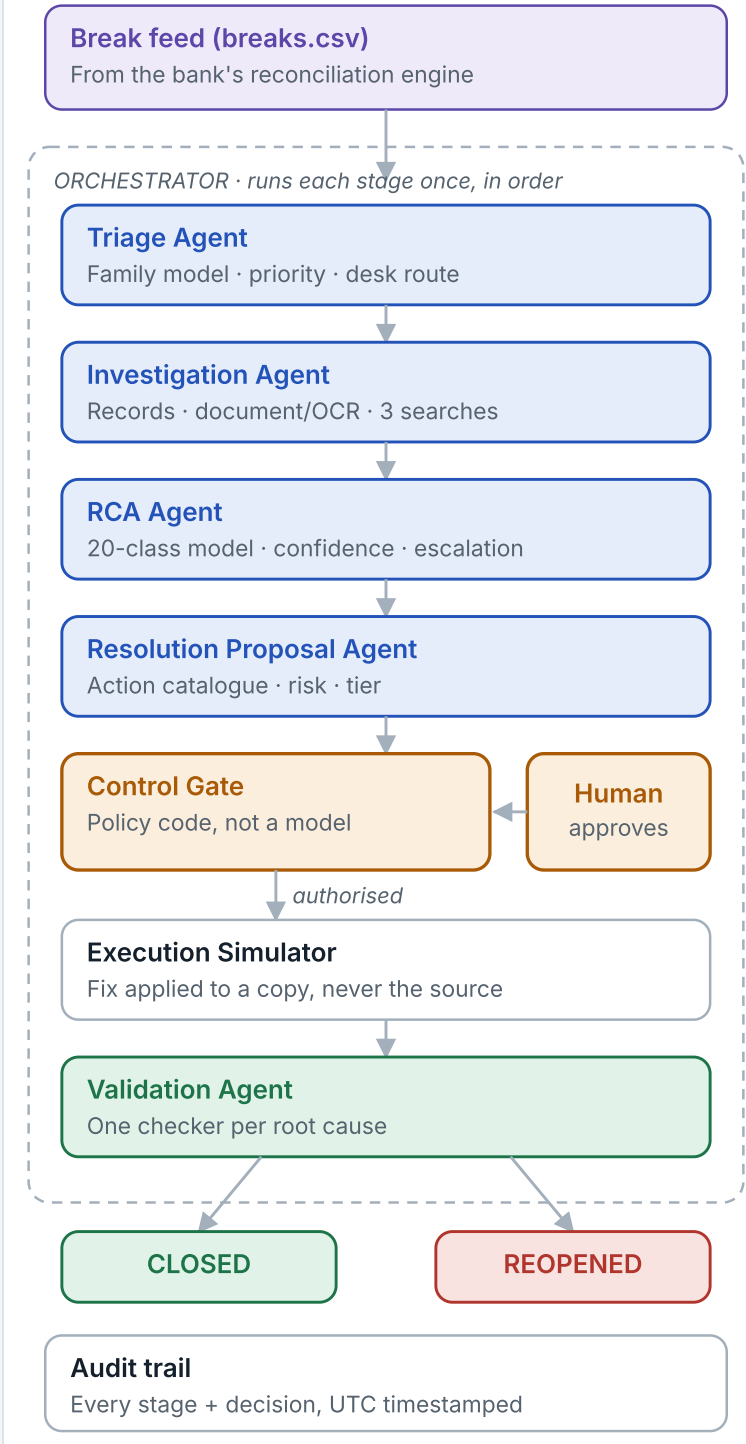
Resolvia starts after Level 0. It never rebuilds matching.

Where Resolvia sits. Level 0 is upstream; Levels 1 and 2 are the baselines it is measured against.

02 · ARCHITECTURE

Six agents, one gate, one LangGraph orchestrator

The Orchestrator is a LangGraph state graph. The case is the graph's state, each agent is a node, and conditional edges route each case by what the agents found: a routine case runs straight down the line, and a case with unusual signals goes to the Investigator after RCA. The Orchestrator owns exactly one decision, which is routing. Every agent has a fixed authority limit. Only the Control Gate, which is plain policy code, can let an action through, and only a recorded human approval opens it for consequential actions.

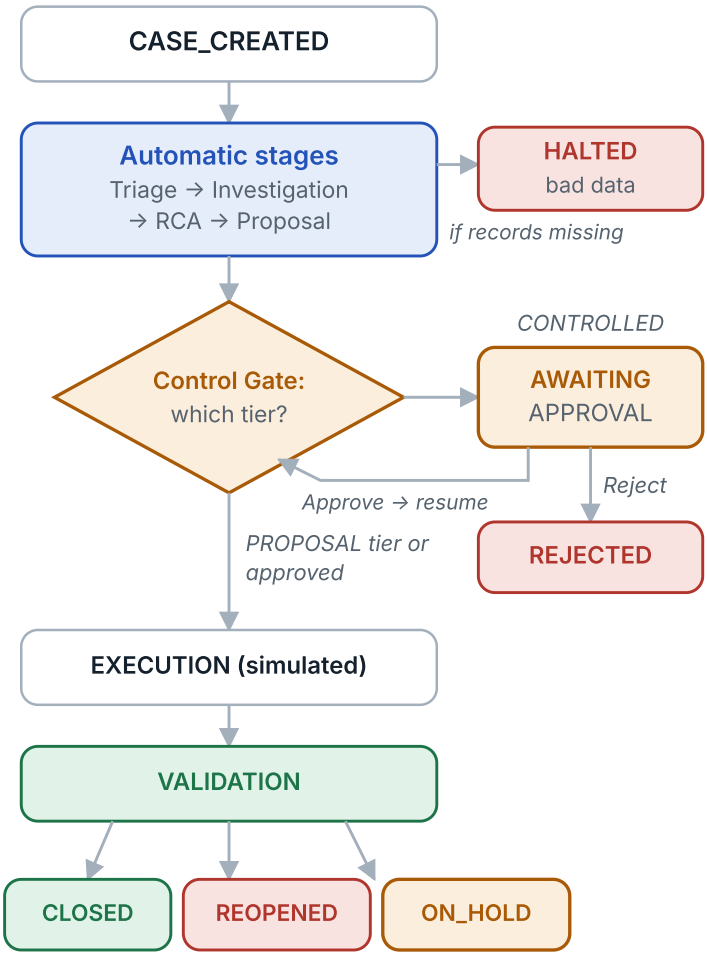


One break, top to bottom. The dashed box is the Orchestrator; everything inside it runs automatically until the gate.

The case lifecycle

Approval is a LangGraph interrupt. `create_case()` runs the graph until the gate blocks, and the graph

pauses there with the case as AWAITING_APPROVAL. Once a person records a decision, `resume_case()` resumes the graph at the gate. A checkpointer keeps every paused run, and every run is saved to SQLite.



Every status a case can reach. A REOPENED case goes back for re-investigation, never a blind retry.

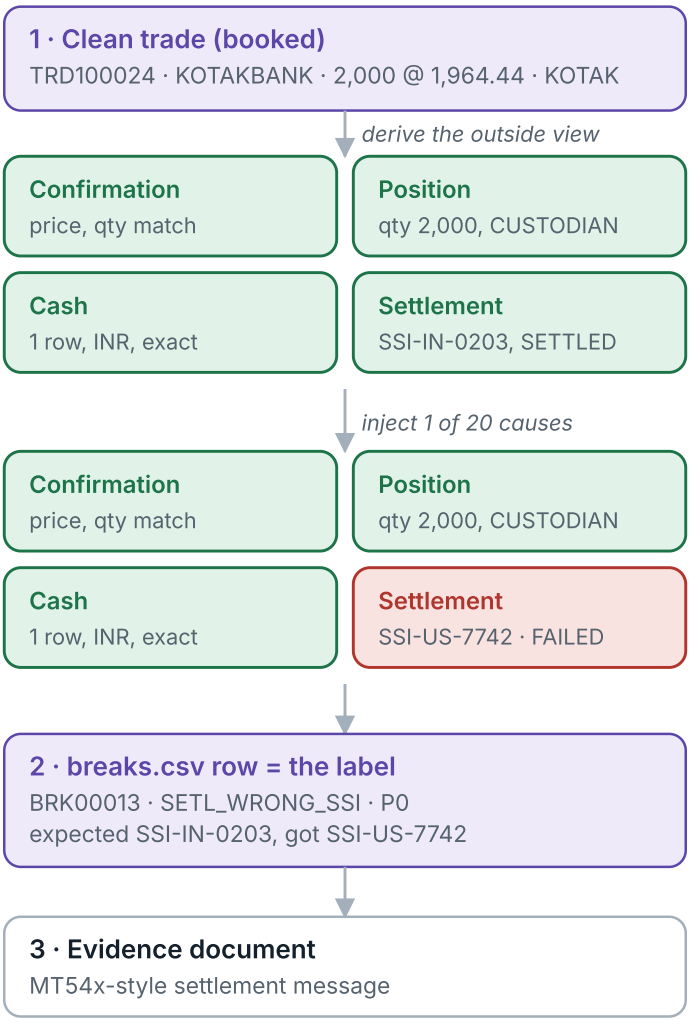
03 · THE DATA

A synthetic bank, built so the answer is known

No public dataset of real trade breaks exists; trade data is confidential everywhere. So, like PaySim

for fraud, I simulated clean operations, injected one known fault per break, and kept the ground truth. Everything is seeded (`random.seed(42)`), so every run is identical.

6,000 trades	**3,000** breaks	**150** per root cause
2,160 documents	**30** scanned images	



How one break is born: start clean, perturb exactly one table, write the label row, write the document.

A note on the data: I set the break rate at 50%, far higher than real life (about 1 to 5%), to give each of the 20 classes 150 examples. Trade breaks change the trade's own `confirmed_*` columns, so the model never has to read the label to find them.

The taxonomy: 4 families, 20 root causes

Family is for routing to the right desk. Root cause is what the model predicts. Severity P0 means settlement or critical risk.

POSITION	6
P0	Missing booking
P1	Wrong quantity
P2	Timing difference
P1	CA incorrect entitlement
P1	CA mandatory not reflected
P1	CA voluntary decision

CASH	4
P0	Missing entry
P1	Duplicate entry
P1	Wrong amount
P2	FX / currency

SETTLEMENT	4
P0	Wrong SSI
P0	Missing instruction
P1	Late counterparty
P2	Incorrect date

TRADE	6
P0	Confirmation missing
P1	Quantity mismatch
P1	Counterparty ID mismatch
P1	Confirmation disagreement
P2	Price mismatch
P2	Date mismatch

The documents

Every corporate-action, settlement and trade break gets a document, plus a 25% sample of position and cash breaks. Each family gets the format a bank would really receive:

Corporate action **SWIFT MT564 notice**

```
:22F::CAEV//BONU
:22F::CAMV//MAND   ← mandatory or VOLU
:22F::CAON//3:1
```

Settlement **SWIFT MT54x message**

```
:16R:GENL
:20C::SEME//…
:95Q: SSI / custodian
```

Trade **NSE contract note**

```
UCC · SEBI reg no.
Qty · Price · STT · GST
(the counterparty's version)
```

Position / Cash **Ops messages**

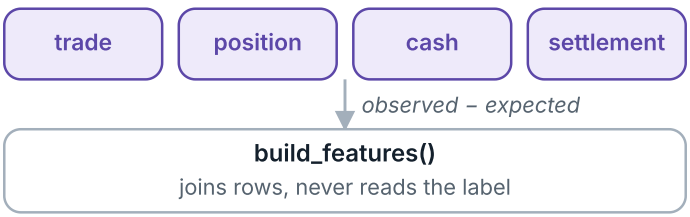
```
email · note · chat · WhatsApp
"RIL" / "Reliance Industries"
dates in 3 formats
```

---

04 · MODELING

Two Random Forests on 15 features

The family model (4 classes) routes in Triage. The root-cause model (20 classes) diagnoses in RCA. Both read the same 15 features, each measuring how far the outside world is from the booking.



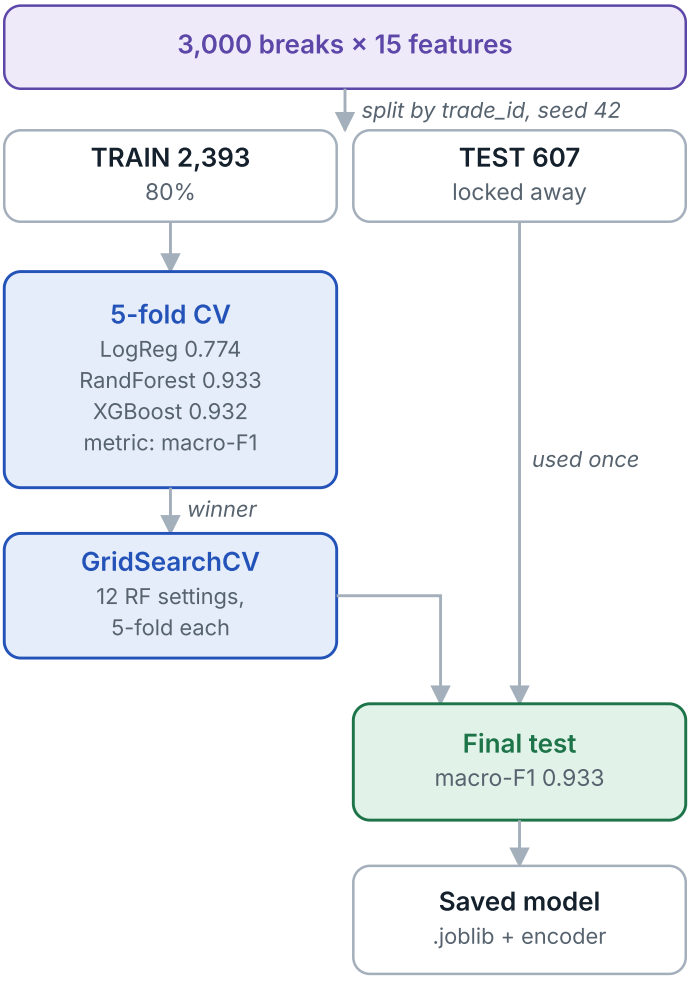
Features come from joining the four tables. The label columns are never used as inputs.

Example: the 15 features for the wrong-SSI break above. Only the cells that differ from a clean trade light up.

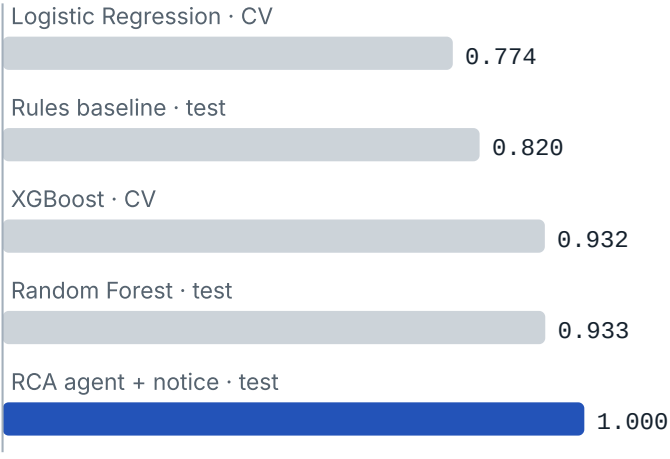
qty_delta_pct	0	position_date_delta	0
cash_row_count	1	cash_amount_delta_pct	0
cash_currency_mismatch	0	settlement_date_delta	0
ssi_mismatch	1	instruction_missing	0
settlement_failed	1	settlement_pending	0
price_delta_pct	0	confirmed_qty_delta_pct	0
date_mismatch	0	counterparty_mismatch	0
confirmation_missing	0		

Why root cause, not family, is the target

I tested both first. Family scored a perfect 1.000 with tree models: it's basically “which table has a non-zero difference”, so there's nothing to compare or tune. Root cause is a real 20-class problem, so it became the graded model. Family is still reported by mapping each predicted cause to its family (also 1.000).



Model comparison and tuning. The test set is touched exactly once, at the end.



Bars start at 0. Macro-F1 across 20 root causes.

Macro-F1 gives every class equal weight, so one failing class can't hide behind good ones.

How 0.933 is made

18 classes are perfect. The two that aren't are a trap I built on purpose. (18 × 1.00 + 0.66 + 0.00) ÷ 20 = 0.933.

pos missing booking	1.00	pos wrong quantity	1.00
pos timing difference	1.00	ca incorrect entitlement	1.00
ca mandatory not reflected	0.66	ca voluntary decision	0.00
cash missing entry	1.00	cash duplicate entry	1.00
cash wrong amount	1.00	cash fx input difference	1.00
setl wrong ssi	1.00	setl late counterparty	1.00
setl missing instruction	1.00	setl incorrect date	1.00
trade price mismatch	1.00	trade quantity mismatch	1.00
trade date mismatch	1.00	trade counterparty id mismatch	1.00
conf missing	1.00	conf disagreement	1.00

The corporate-action trap

Mandatory and voluntary corporate actions were generated to look identical in the numbers. The model calls both “mandatory”. Only the notice tells them apart, and the actions are opposite.

Mandatory split CA_MANDATORY_NOT_REFL.	Voluntary tender CA_VOLUNTARY_DECISION
Booked qty	
500	500
Custodian qty	
500	500
All 15 features	
zero	zero
Model predicts	
MANDATORY	MANDATORY
MT564 notice	
CAMV//MAND	CAMV//VOLU
Right action	
Apply ratio	HOLD for client

Numbers can't tell them apart. The document can.

This is the project's main argument: structured ML has a ceiling exactly where a business judgment lives.

So the RCA agent reads the notice. Its CAMV field, event code and narrative, each read with tolerance for an OCR slip, override the model, which takes root-cause macro-F1 from 0.933 to 1.000 on the test set. A person still signs off every corporate action, because **confidence never decides authority.**

---

05 · THE AGENTS

Each agent: one question, one limit

An “agent” here is a Python module with one entry function, a fixed input and output, and a stated authority limit. Five follow a fixed procedure, each

with the cheapest reliable tool for its step. The sixth, the Investigator, is an LLM agent that plans its own steps for the unusual cases.

Triage
What is it, where does it go?
`No financial action`

Investigation
What evidence exists?
`Read only`

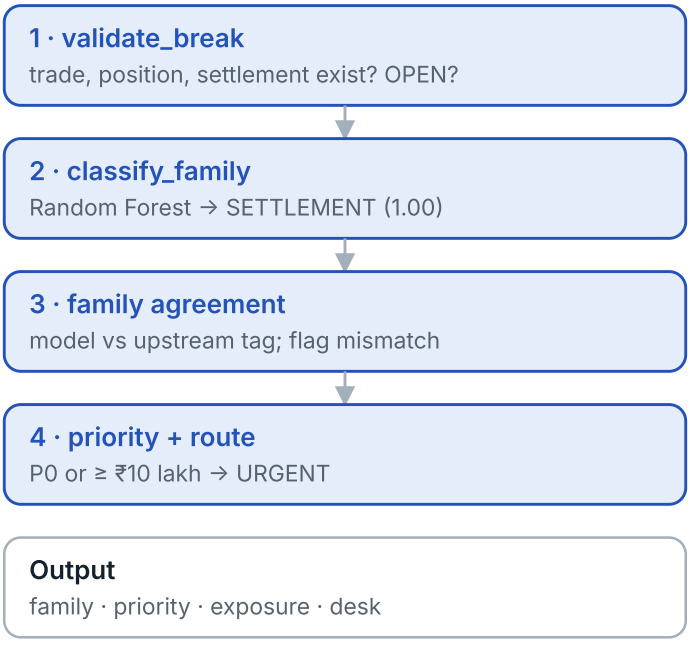
RCA
Why did it happen?
`Recommends only`

Investigator
What else should I check?
`Read only, unusual cases`

Resolution Proposal
What should we do?
`Proposes only`

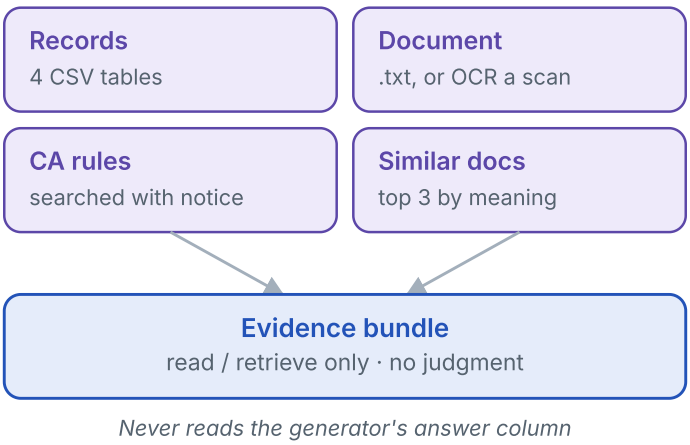
Validation
Did the fix work?
`CLOSE or REOPEN`

Triage Agent



Data check, family model, sanity check against the upstream tag, then priority and desk. Corporate actions always go to the CA specialist queue.

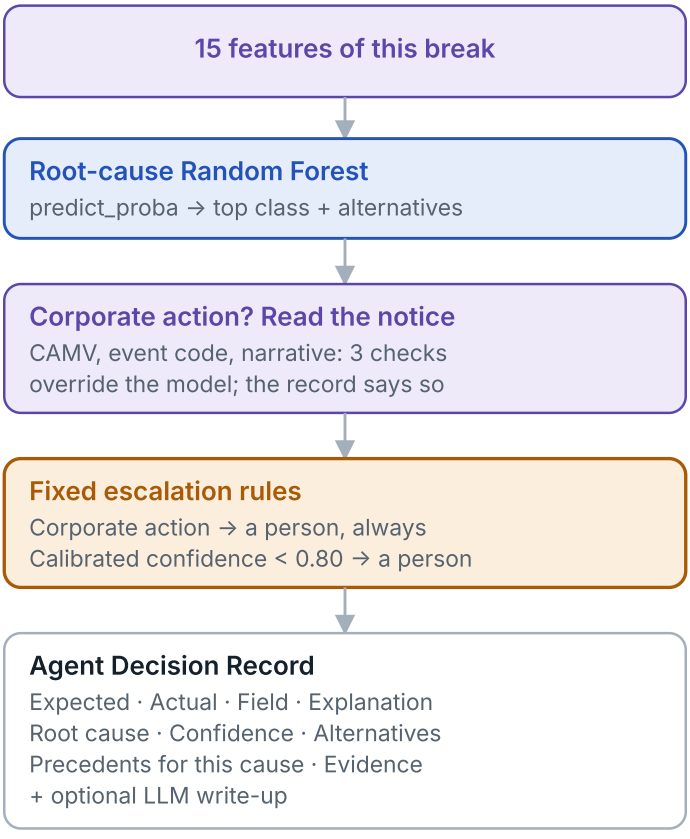
Investigation Agent



Never reads the generator's answer column

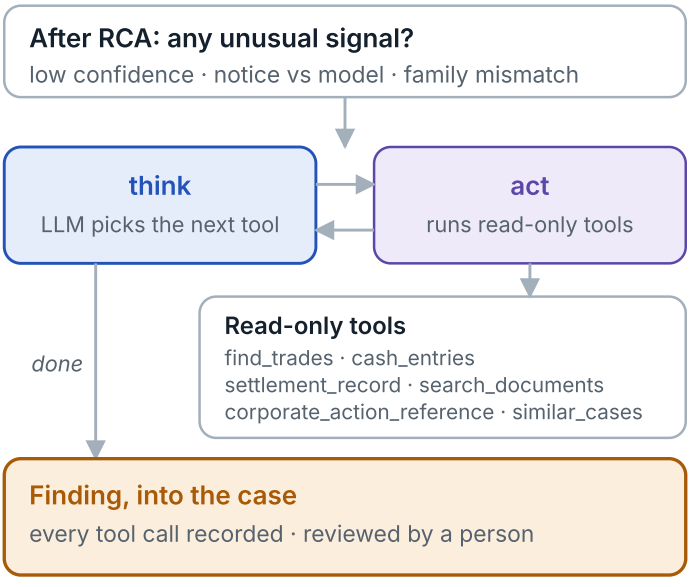
Gathers, never judges. It only sees what a reconciliation engine would hand over.

RCA Agent



The model diagnoses, the notice corrects it where it can, and fixed rules decide whether a person reviews it. Precedents are looked up for the diagnosed cause.

Investigator Agent



The Investigator is its own small LangGraph loop, up to three rounds of tool calls.

Routine breaks are fully explained by the evidence Investigation gathered, so they go straight to a proposal. When a case shows an unusual signal, the graph sends it here first. The Investigator looks for the trade under other keys, related cash and settlement entries, documents, corporate action definitions and past cases, decides what to check next from what it finds, and writes a short finding for the reviewer. 69 of the 607 test breaks carry at least one signal.

Resolution Proposal Agent

Once the cause is known, the fix is mechanical, so it is looked up in a fixed catalogue rather than invented by an LLM. Each entry has an action, an expected outcome and an authority tier. Risk is arithmetic: HIGH if P0 or exposure ≥ ₹10 lakh.

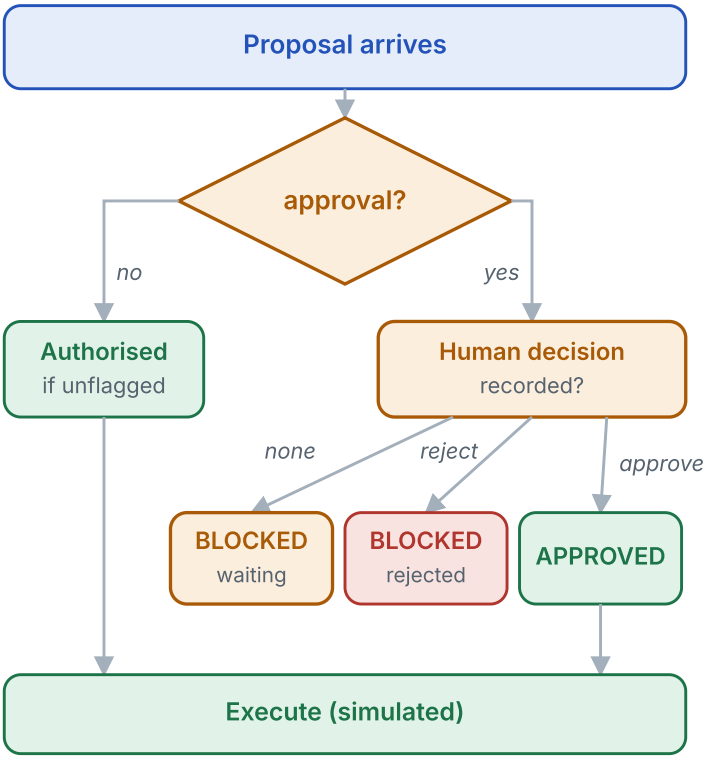
15 causes · ~75%
CONTROLLED_ACTION · human approval

5 · ~25%
PROPOSAL

PROPOSAL tier (administrative): timing difference, late counterparty, settlement date, trade date, missing confirmation. Everything that changes money, an SSI or a counterparty is CONTROLLED. Voluntary corporate action = **HOLD**. Even a PROPOSAL-tier step needs approval if RCA flagged the diagnosis.

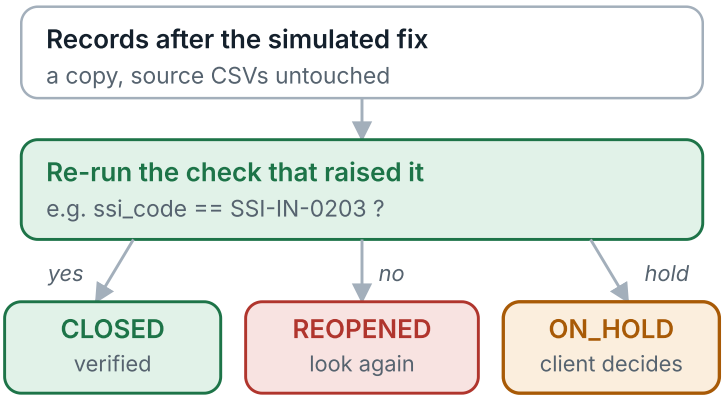
Two different flags. RCA's `requires_human_decision` asks “is the diagnosis uncertain?”. Resolution's `approval_required` asks “does the action need sign-off?”. A confident diagnosis can still need approval.

Control Gate



record_decision() is the only door a human approval comes through; no code can approve itself. Result: 0 unauthorised executions.

Validation Agent

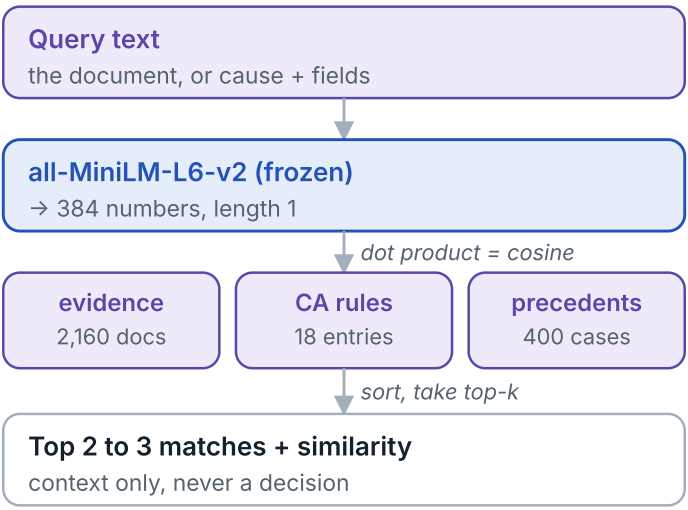


Doesn't trust the diagnosis: a wrong fix reopens
Gate blocked = nothing ran = stays OPEN

Validation re-runs the reconciliation on the field that broke. If the diagnosis was wrong, the wrong fix runs and this catches it.

06 · RETRIEVAL, OCR, LLM

Context comes from search, not memory



One frozen embedding model, three local indexes, plain NumPy. No vector database needed at this size.

Precedents

20 breaks per root cause were run through the real pipeline: 400 cases: 380 CLOSED and 20 ON_HOLD. The 20 on hold are voluntary corporate actions waiting for the client, which is the right outcome.

OCR

EasyOCR reads the 30 scans, ~99% accurate on key tokens (trade IDs, symbols, quantities). It runs only when a document exists solely as an image.

LLM (optional)

OpenRouter with a fallback list of free models. Writes the RCA narrative, extracts fields from emails, summarises uploads. It explains; it never decides.

---

07 · WALKTHROUGH

One P0 break, start to finish

BRK00013, SETL_WRONG_SSI: the payment went to the wrong settlement account and failed. This is the case I use to demo it.

Case created
Analyst opens the break; CASE_CREATED is logged.

Triage
Records OK · family SETTLEMENT (agrees) · P0 → URGENT · Settlements Desk.

Investigation
Settlement row: SSI-US-7742, FAILED, INCORRECT. MT54x document read. 3 similar documents found.

RCA
SETL_WRONG_SSI at ~100%. Expected SSI-IN-0203, actual SSI-US-7742. Not CA, confident → no forced review. 3 precedents pulled.

Proposal
“Correct the SSI and resubmit.” CONTROLLED_ACTION, risk HIGH.

Gate blocks
No approval yet → AWAITING_APPROVAL. Nothing touched.

Human approves
Approve + note. Who and when are recorded; the case resumes.

Execution

On a copy: correct SSI, INSTRUCTED, SETTLED.

Validation

Re-checks ssi_code: now SSI-IN-0203 → CLOSED.

Audit

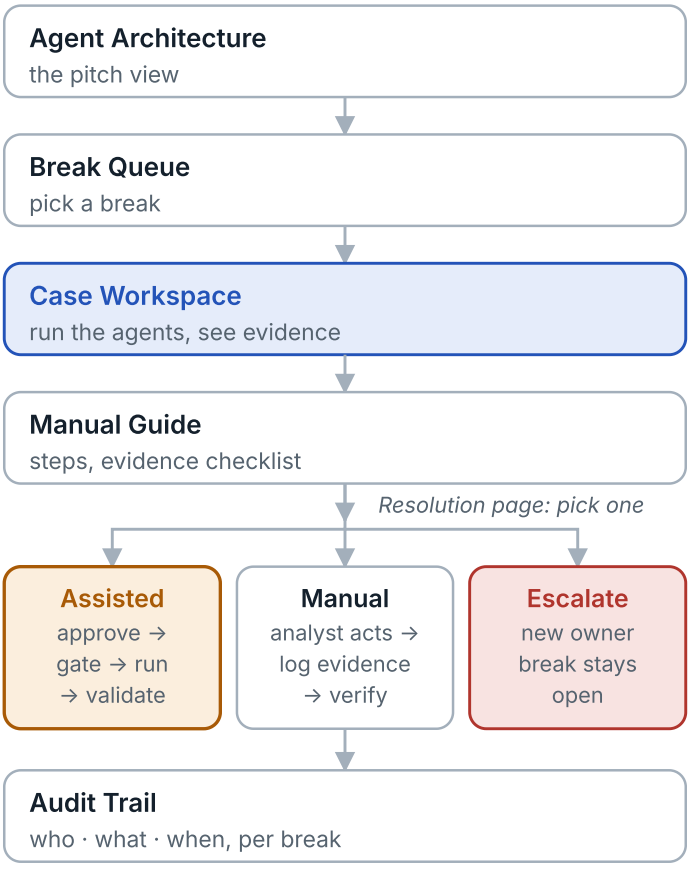
Every step above is in the trail with a timestamp and Operations ID.

---

08 · THE APP

Streamlit workbench for three people

Analyst: next action and evidence. Manager: workload, ageing, exposure. Auditor: who did what, when. The app calls the backend in-process through `backend_bridge.py`; the operator ID (default OPS-1047) is stamped on every event.



The demo path through the app. Escalation transfers ownership; it is never a closure.

Trying it yourself. The first analysis takes about 20 seconds while the models load. I'd start with BRK00013 end to end, then BRK00030, a voluntary corporate action, where the notice overrides the model and the case ends on hold. Cases, approvals and the audit trail are saved, so they survive a restart.

09 · RESULTS

The numbers

1.000

0.933
model alone (test)

RCA agent with the notice (test)

0.820
rules baseline

77%
test breaks needing a person

572 / 35
test breaks closed / on hold

0
unauthorised actions

0.973
agent F1 on the messiest stress test

4.89
sources per break, manual

The three-way comparison

Level 1

Manual

Analyst opens about 5 evidence sources per break (4 tables, plus the document, plus the CA notice). A process proxy, not a made-up stopwatch time.

Level 2

Rules / RPA

If-else rules on the same 15 features: 0.820. Nails mechanical causes, fails on corporate actions.

Level 3

Resolvia

0.933 from the model, 1.000 once the notice is read, plus evidence assembly, precedents, enforced approval, simulated execution and a re-check before closing.

Business value formula (no invented dollar figure): capacity released = break volume × manual minutes per break × automation coverage × working days. Cost avoided = capacity × loaded analyst cost per hour.

10 · STRESS TESTS

What happens when the data gets messy

A perfect score on clean synthetic data proves the plumbing, not the model. So I added noise to prices and cash, gave some trades a second fault, and damaged or removed the corporate action notices, then measured where things bend.

Clean data

Model 0.933 · agent 1.000

The starting point.

1% noise on prices and cash

Model 0.866 · agent 0.924

A clean-trained model falls to 0.820 here.

Everything at once

Model 0.937 · agent 0.973

Noise, two-fault trades, scanned notices with 5% of characters misread, a quarter of notices missing.

Accuracy bends, safety doesn't. In every scenario, wrong diagnoses either reached a person or failed Validation's re-check and reopened. Wrong fixes that ran unnoticed: 0.

Three changes to the model

- **Train on messy copies**
  Three perturbed copies of the training breaks. Keeps the model 4 to 5 points ahead at the worst noise level, at no cost on clean data.
- **Calibrate the confidence**

86% of the raw forest's confidences were exactly 0 or 1. After isotonic calibration, calibration error on messy data falls from 0.097 to 0.024.

Pick the threshold by cost

Review = 1, reopened fix = 5, unseen error = 50. 0.80 was cheapest: 23 more reviews than 0.50, 9 fewer reopened fixes.

Reading damaged notices

Reading only the CAMV field falls to 65% accuracy on corporate actions when 30% of characters are misread. Cross-checking the event code and the narrative keeps it at 96%. With no notice at all it drops to 65%, which is why every corporate action still goes to a person.

How much review is enough?

`strict (live)`

81% reviewed · 1 reopened · 0 unseen errors

`auto under 5 lakh`

72% reviewed · 12 reopened · 0 unseen errors

`auto under 10 lakh`

68% reviewed · 17 reopened · 0 unseen errors

Messy data. Auto-run only for non-P0, non-corporate-action fixes with confidence of at least 0.95. Less review buys more rework, not hidden errors. I kept the live gate strict: where to set that limit is the bank's call.

Measuring the Gen AI layer

I score document extraction field by field against ground truth on 1,162 trades in emails, chats, WhatsApp messages, batch emails and contract notes. A rules-only baseline reads clean text

almost perfectly and falls to about a third on prices and dates once 5% of characters are misread. That's the gap the LLM is there to close, and the same script scores it.

Also new: 23 automated tests on every push, and cases, approvals and the audit trail saved in SQLite.

---

11 · RESPONSIBLE AI

Governance is part of the design

In operations a wrong automated action is a failed payment, so I built governance into the workflow from the start. Each principle maps to code in the repo.

Human oversight

Anything that moves money, an SSI or a counterparty waits for a person. Corporate actions always do.

Evidence: 77% of test breaks waited

Accountability

A named approver and a note on every decision, through one door: record_decision().

Evidence: 0 unauthorised actions

Transparency

Each diagnosis shows expected, actual, confidence, alternatives and the evidence it used.

Evidence: notice overrides are logged

Reliability

Fixes run on copies. Validation re-checks the field, so a wrong diagnosis reopens the case.

Evidence: 0 unseen wrong fixes under stress

Privacy

Synthetic data, no customer details. Uploaded files stay in memory and are never stored.

Evidence: no real PII anywhere

Bounded GenAI

The LLM only explains or extracts. It is optional and never allowed to decide.

Evidence: works without an API key

- **Confidence never decides authority.** Even a 100% confident diagnosis needs approval for a controlled action.
- **Corporate actions always go to a person,** even after the notice is read, because a real notice can be late or missing.
- **NIST AI RMF:** authority tiers and one approval door (Govern), scoped agents and the corporate-action failure mode (Map), baselines, a one-time test set, stress tests and calibration (Measure), approval before and re-checks after every action (Manage).
- **Not covered yet:** fairness across counterparties, drift monitoring, and real sign-in for approvers.

The root-cause model also has a model card in the repo: docs/model-card.md.

---

12 · QUESTIONS

Limits and common questions

What my build doesn't do

- Cases live in a local SQLite file; a bank would want a managed database with an append-only trail.
- Execution is simulated on a copy; no real system is touched.
- Data is synthetic; scores outside the corporate-action classes are near-perfect because synthetic differences are clean.

Choices I made so it behaves like a real system

- **It never reads the answer.** The root-cause columns in breaks.csv are ground truth for training only. The app shows what the reconciliation engine reported until RCA has diagnosed the break.
- **Fixes follow the diagnosis.** Execution applies the fix for the predicted cause, and Validation re-checks the field that broke, so a wrong diagnosis reopens instead of closing.
- **The notice settles the trap.** RCA reads three signals from the MT564 (CAMV, event code, narrative) and overrides the model only when they agree.
- **Flags reach the gate.** Even an administrative step needs approval if RCA flagged the diagnosis.
- **Voluntary means hold.** These end ON_HOLD, waiting for the client's election, not REOPENED.

Questions I get asked

▶ **Isn't 1.000 too good to be true?**

▶ **Why synthetic data?**

▶ Why root cause instead of family as the target?

▶ Why Random Forest over XGBoost?

▶ Where does the 0.80 threshold come from?

▶ Why macro-F1, not accuracy?

▶ Where is the “agentic” part if the agents aren't LLMs?

▶ Why LangGraph?

▶ What stops the AI from changing a payment?

▶ If the notice fixes it, why still send corporate actions to a person?

▶ What does RAG do here?

▶ How do you know a fix worked?

▶ How would this go to production?

---

13 · WHERE THINGS LIVE

Repository map

`data/`

generators, taxonomy, features, 5 CSVs, 2,160 docs, 30 scans

`backend/agents/`

triage, investigation, rca, resolution, validation (+ ingestion)

`backend/`

orchestrator (LangGraph), control_gate, execution_simulator, audit, llm

`backend/models/`

training scripts + saved .joblib models

`backend/embeddings/`

MiniLM store + 3 indexes

`backend/knowledge/`

18 CA rules, 400 precedent cases

`backend/eval/`

baselines, end-to-end and stress tests, approval policy, extraction

`backend/store.py`

SQLite for cases, decisions and the audit trail

`tests/`

28 tests, run on GitHub Actions

`backend/api/`

REST API: open a case, record a decision

`docs/`

the tour, diagrams, model report, EDA notebook, proposal

`frontend/`

Streamlit pages and backend bridge

Stack: Python, Streamlit, Plotly, pandas, scikit-learn, XGBoost (compared), sentence-transformers, EasyOCR,

FastAPI, OpenRouter. Live app: resolvia.streamlit.app · Code: github.com/hiralsarkar/Resolvia

Hiral Sarkar · github.com/hiralsarkar/Resolvia. Colours: blue = AI agent, amber = human control, green = verified, red = failure, violet = data.